

RCA8_LLM sim out

```
VCD info: dumpfile RCA8_tb.vcd opened for output.
===== RCA8 TEST START =====
PASS | a=00000000 b=00000000 | sum=00000000 cout=0 | c=00000000
PASS | a=11111111 b=11111111 | sum=11111110 cout=1 | c=11111111
PASS | a=00000001 b=11111111 | sum=00000000 cout=1 | c=11111111
PASS | a=10101010 b=01010101 | sum=11111111 cout=0 | c=00000000
PASS | a=00001111 b=00000001 | sum=00010000 cout=0 | c=00011111
PASS | a=00100100 b=10000001 | sum=10100101 cout=0 | c=00000000
PASS | a=00001001 b=01100011 | sum=01101100 cout=0 | c=00000011
PASS | a=00001101 b=10001101 | sum=10011010 cout=0 | c=00011011
PASS | a=01100101 b=00010010 | sum=01110111 cout=0 | c=00000000
PASS | a=00000001 b=00001101 | sum=00001110 cout=0 | c=00000001
===== TEST COMPLETE =====
D:/advanced_project/FinalProject_adder/RCA8_LLM_tb.v:87: $finish called at 100000 (1ps)
```

KSA8_LLM

```
VCD info: dumpfile ksa8.vcd opened for output.
Time | a | b | g | p | g1 | p1 | g2 | p2 | g3 | p3 | sum | cout
0 | 00000000 00000000 | 00000000 00000000 | 00000000 00000000 | 00000000 00000000 | 00000000 00000000 | 00000000 00000000 | 00000000 0
PASS: a=0 b=0 => sum=0
10000 | 11111111 00000001 | 00000001 11111110 | 00000011 11111100 | 00001111 11110000 | 11111111 00000000 | 00000000 1
PASS: a=255 b=1 => sum=0
20000 | 11111111 11111111 | 11111111 00000000 | 11111111 00000000 | 11111111 00000000 | 11111111 00000000 | 11111110 1
PASS: a=255 b=255 => sum=254
30000 | 00000001 00000001 | 00000001 00000000 | 00000001 00000000 | 00000001 00000000 | 00000001 00000000 | 00000010 0
PASS: a=1 b=1 => sum=2
40000 | 10000000 10000000 | 10000000 00000000 | 10000000 00000000 | 10000000 00000000 | 10000000 00000000 | 00000000 1
PASS: a=128 b=128 => sum=0
50000 | 00100100 10000001 | 00000000 10100101 | 00000000 00000001 | 00000000 00000001 | 00000000 00000001 | 10100101 0
PASS: a=36 b=129 => sum=165
60000 | 00001001 01100011 | 00000001 01101010 | 00000011 01000000 | 00000011 00000000 | 00000011 00000000 | 01101100 0
PASS: a=9 b=99 => sum=108
70000 | 00001101 10001101 | 00001101 10000000 | 00001101 00000000 | 00001101 00000000 | 00001101 00000000 | 10011010 0
PASS: a=13 b=141 => sum=154
PASS: a=36 b=129 => sum=165
60000 | 00001001 01100011 | 00000001 01101010 | 00000011 01000000 | 00000011 00000000 | 00000011 00000000 | 01101100 0
PASS: a=9 b=99 => sum=108
70000 | 00001101 10001101 | 00001101 10000000 | 00001101 00000000 | 00001101 00000000 | 00001101 00000000 | 10011010 0
PASS: a=36 b=129 => sum=165
60000 | 00001001 01100011 | 00000001 01101010 | 00000011 01000000 | 00000011 00000000 | 00000011 00000000 | 01101100 0
PASS: a=9 b=99 => sum=108
PASS: a=36 b=129 => sum=165
60000 | 00001001 01100011 | 00000001 01101010 | 00000011 01000000 | 00000011 00000000 | 00000011 00000000 | 01101100 0
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
PASS: a=36 b=129 => sum=165
60000 | 00001001 01100011 | 00000001 01101010 | 00000011 01000000 | 00000011 00000000 | 00000011 00000000 | 01101100 0
PASS: a=9 b=99 => sum=108
70000 | 00001101 10001101 | 00001101 10000000 | 00001101 00000000 | 00001101 00000000 | 00001101 00000000 | 10011010 0
PASS: a=13 b=141 => sum=154
80000 | 01100101 00010010 | 00000000 01110111 | 00000000 01100111 | 00000000 00000111 | 00000000 00000111 | 01110111 0
PASS: a=101 b=18 => sum=119
90000 | 00000001 00001101 | 00000001 00001100 | 00000001 00001000 | 00000001 00000000 | 00000001 00000000 | 00001110 0
PASS: a=1 b=13 => sum=14
100000 | 01110110 00111101 | 00110100 01001011 | 01111100 00000011 | 01111100 00000011 | 01111100 00000011 | 10110011 0
PASS: a=118 b=61 => sum=179
110000 | 11101101 10001100 | 10001100 01100001 | 10001100 01000001 | 10001100 00000001 | 10001100 00000001 | 01111001 1
PASS: a=237 b=140 => sum=121
120000 | 11111001 11000110 | 11000000 00111111 | 11000000 00111111 | 11000000 00111111 | 11000000 00111111 | 10111111 1
PASS: a=249 b=198 => sum=191
130000 | 11000101 10101010 | 10000000 01101111 | 10000000 01001111 | 10000000 00001111 | 10000000 00001111 | 01101111 1
PASS: a=197 b=170 => sum=111
140000 | 11100101 01110111 | 01100101 10010010 | 11100111 00000000 | 11100111 00000000 | 11100111 00000000 | 01011100 1
PASS: a=229 b=119 => sum=92
Simulation Finished.
```

Adderbest1(optimized from RCA8)

```

=====
start 8bits_adder test...
=====
VCD info: dumpfile adder8_sim.vcd opened for output.

=====
test conclusion: [ PASS ]
passed      131072 test!
=====

```

Bestadder2(optimized from KSA8)

```

=====
start tsetting 8bits_adder (Cin initial = 0)...
=====
VCD info: dumpfile adder8_sim.vcd opened for output.

=====
conclusion: [ PASS ]
passed      65536 test!
=====

```

Yosys equivalent check for adderbest1(from RCA8)

```

Found 2 mismatched outputs:
  \sum [7:0]
  \cout

Equivalence successfully proven:      0
Equivalence failed:                   1
ERROR: Equivalence check failed!

```

Yosys equivalent check for adderbest1(from KSA8)

```
yosys> equiv_simple
4. Executing EQUIV_SIMPLE pass.
Performing SAT check on matching outputs...
- Output \sum: Proven equivalent (8 bits).
- Output \cout: Proven equivalent (1 bit).

yosys> equiv_status
5. Executing EQUIV_STATUS pass.
Found 0 mismatched outputs.
Found 9 matched outputs.

Equivalence successfully proven!
```

1. Analysis of Equivalence Check Failure (adderbest1.v)

The initial attempt to verify the adderbest1.v design against the golden model (RCA8.v) failed primarily due to **Interface and Structural Mismatches** rather than fundamental logical errors.

- **Port Inconsistency:** The golden reference model (RCA8.v) defines an 8-bit adder with 4 ports: a, b, sum, and cout. However, adderbest1.v included an additional input port, cin. Yosys requires an exact match of input/output signals to perform a formal comparison; the extra cin signal prevented the tool from mapping the two designs.
- **Module Naming Conflict:** The script attempted to rename a module called adder8, but the Verilog file defined the module as RCA8. This caused the Yosys script to terminate during the rename command before it could reach the logic verification stage.
- **Sub-module Redundancy:** Both the reference file and the generated file contained definitions for a full adder (FA). When loaded simultaneously, this created namespace conflicts in the Yosys database, leading to "Invalid number of arguments" errors during the equiv_make process.

2. Final Optimized Architecture: Kogge-Stone Adder (KSA)

The final deliverable, adderbest2.v, implements a **Kogge-Stone Adder (KSA)**

architecture. While `adderbest1.v` also utilized a parallel-prefix approach, `adderbest2.v` was successfully optimized to pass formal verification by aligning its interface with the project requirements (removing the external `cin` and hardcoding the internal carry-in to zero).

Architectural Implementation:

1. **Pre-Processing:** Generation of bit-wise Propagate (p_i) and Generate (g_i) signals.
 2. **Parallel Prefix Network:** A three-stage tree structure with spans of 1, 2, and 4 to compute group carries in parallel.
 3. **Post-Processing:** Final sum generation through the XOR of propagate signals and resolved carries
-

3. Rationale for Selecting the KSA Architecture

The Kogge-Stone Adder was selected over the traditional Ripple Carry Adder (RCA) for the final optimization for the following reasons:

- **Logarithmic Delay:** In an 8-bit RCA, the carry must ripple through 8 stages, resulting in $O(N)$ delay. The KSA reduces this to $O(\log_2 N)$ stages (3 stages for 8 bits), significantly increasing the maximum clock frequency of the circuit.
- **Performance Reliability:** KSA is widely regarded as one of the fastest adder architectures because it minimizes the logic depth of the carry generation network.
- **Structural Regularity:** Despite the increased wiring complexity, the KSA's uniform structure makes it an ideal candidate for demonstrating the LLM's capability to handle complex, highly-indexed parallel logic.
- **PPA Trade-off:** The selection demonstrates a deliberate "Power-Performance-Area" trade-off, where a slight increase in area and wiring density is accepted to achieve superior timing performance.